

Automatização de Configurações - IaC

Infraestruturas de Computação na Nuvem, Aula 07

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda (1/2)

- DevOps: a tensão entre desenvolvimento e operações.
- Motivação: dos scripts manuais à automatização.
- Infrastructure as Code (IaC).
- Declarativo vs imperativo; idempotência.
- Gestão de configuração: Ansible vs Puppet/Chef.
- Estrutura de um projeto Ansible.

Agenda (2/2)

- Provisionamento declarativo com Terraform.
- Gestão do ficheiro de estado.
- Configuração vs provisionamento.
- Infraestrutura mutável vs imutável.
- Gestão de segredos.
- CI/CD e GitOps aplicados a infraestrutura.

DevOps: a Tensão Entre Desenvolvimento e Operações

- Historicamente, as equipas de **Dev** (criar e evoluir software) e **Ops** (manter a infraestrutura estável) têm objetivos em tensão: uma quer mudar depressa, a outra quer que nada quebre.
- Sintoma clássico: “funciona na minha máquina” (Dev) contra “não é o servidor, é o teu código” (Ops) – cada equipa desconfia da outra.
- Resposta tradicional (burocracia, processos rígidos, aprovações manuais) tende a piorar o problema: mais lentidão, sem mais qualidade.
- **DevOps**: cultura e conjunto de práticas que aproxima as duas equipas, apoiada em infraestrutura flexível, versionamento, integração contínua e automatização – o tema desta aula.

O Problema da Configuração Manual

- Configurar servidores à mão não escala além de algumas máquinas.
- **Configuration drift**: cada servidor evolui de forma diferente ao longo do tempo.
- Erro humano: passos esquecidos, comandos mal aplicados, falta de consistência.
- Falta de reprodutibilidade: difícil recriar um ambiente igual a outro.

Consequências da Falta de Automatização

- “Funciona na minha máquina”: ambientes de desenvolvimento, teste e produção divergem.
- Recuperação de desastre lenta: recriar infraestrutura manualmente demora horas ou dias.
- Auditoria e conformidade difíceis sem registo do que foi alterado e quando.
- Escalar equipas e sistemas exige processos repetíveis.

Infrastructure as Code (IaC)

- Infraestrutura descrita em ficheiros de texto (código), não em passos manuais.
- Código versionado em Git: histórico, revisão por pares, rollback.
- **Idempotência**: aplicar a mesma definição várias vezes produz sempre o mesmo resultado.
- **Repetibilidade**: o mesmo código gera ambientes idênticos, em qualquer altura.

Vantagens do IaC

- Documentação viva: o código é a documentação da infraestrutura.
- Testabilidade: as mudanças podem ser validadas antes de chegar a produção.
- Redução do tempo de provisionamento de dias para minutos.
- Base para práticas de DevOps e para escalar operações.

Declarativo vs Imperativo

- **Imperativo:** descreve-se a *sequência de passos* a executar (ex. um script shell: instalar, copiar, reiniciar).
- Problema: o resultado depende do estado inicial da máquina, e repetir o script pode causar efeitos indesejados.
- **Declarativo:** descreve-se o *estado final pretendido*; a ferramenta determina que ações são necessárias para lá chegar.
- É o mesmo princípio já visto no Kubernetes (Aula 05): declarar o estado desejado e deixar o sistema reconciliar.
- A abordagem declarativa é o que torna a idempotência possível.

Idempotência na Prática

- Uma operação é **idempotente** se aplicá-la várias vezes produz o mesmo resultado que aplicá-la uma só vez.
- Exemplo não idempotente: `echo "linha">> ficheiro.conf` – cada execução acrescenta uma nova linha.
- Exemplo idempotente: "garantir que o ficheiro contém esta linha-- se já lá estiver, nada acontece.
- Consequência operacional: pode-se aplicar o mesmo código repetidamente, com segurança, para corrigir *drift*.
- As ferramentas reportam o que mudou (*changed*) e o que já estava conforme (*ok*), tornando as execuções auditáveis.

Gestão de Configuração: Agentless vs Agent-Based

- Ferramentas de gestão de configuração definem o estado desejado *dentro* de uma máquina já existente.
- **Agentless** (ex. Ansible): liga-se via SSH, sem necessidade de instalar software no host gerido.
- **Agent-based** (ex. Puppet, Chef): requer um agente instalado e em execução em cada host gerido.
- Trade-off: simplicidade e menor overhead vs maior escalabilidade em ambientes muito grandes.

Ansible

- Comunica por SSH; não exige agente no host gerido.
- Playbooks escritos em **YAML**, legíveis e declarativos.
- **Inventory**: ficheiro que lista os hosts geridos, agrupados por função.
- Módulos idempotentes: instalar pacotes, copiar ficheiros, gerir serviços.

Estrutura de um Projeto Ansible

- **Playbook**: ficheiro YAML que associa um conjunto de *tasks* a um grupo de hosts.
- **Task**: uma ação individual, executada por um **módulo** (ex. apt, copy, service).
- **Handler**: task especial, executada apenas se outra task notificar que algo mudou (ex. reiniciar um serviço após alterar a configuração).
- **Role**: forma de organizar tasks, variáveis, templates e ficheiros numa unidade reutilizável.
- **Template Jinja2**: ficheiro de configuração com variáveis, preenchido de forma diferente para cada host.
- **Facts**: informação recolhida automaticamente sobre cada host (SO, memória, IPs), utilizável nas condições e templates.

Puppet e Chef

- Exigem um **agente** instalado em cada host, que comunica periodicamente com um servidor central.
- Puppet: linguagem declarativa própria (manifests); modelo cliente-servidor com verificações periódicas.
- Chef: “recipes” escritas em Ruby (DSL); também assenta em agentes e um servidor central.
- Adequados a parques muito grandes, com necessidade de garantir conformidade contínua.

Ansible vs Puppet vs Chef: Prós e Contras

- **Ansible:** curva de aprendizagem suave (YAML, sem DSL própria), sem agentes – mas menos expressivo que uma linguagem completa e mais lento em parques muito grandes.
- **Puppet:** comunidade madura, interface web completa, mas a linguagem declarativa própria tem uma curva de aprendizagem mais acentuada e o modelo baseado em dependências pode complicar-se em projetos grandes.
- **Chef:** máximo controlo e flexibilidade (as “recipes” são código Ruby real), mas exige saber Ruby e tende a gerar bases de código mais complexas.
- Não há uma escolha universalmente certa: a decisão pesa simplicidade operacional, dimensão do parque e o perfil técnico da equipa.

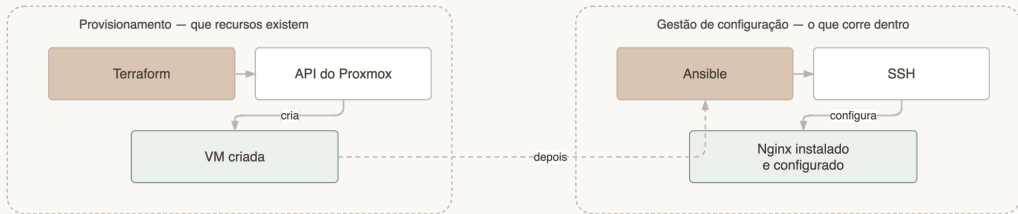
Terraform e Provisionamento Declarativo

- Terraform: ferramenta declarativa para provisionar recursos de infraestrutura (VMs, redes, storage).
- **Providers**: plugins que traduzem código Terraform em chamadas a APIs de clouds ou serviços específicos.
- **Ficheiro de estado (state file)**: regista o que foi efetivamente criado, para comparar com o desejado.
- Fluxo `plan` / `apply`: primeiro mostra as alterações previstas, só depois as executa.

Gestão de Configuração vs Provisionamento

- **Provisionamento:** decide *que* máquinas e recursos existem (ex. Terraform).
- **Gestão de configuração:** decide *o que corre* dentro de cada máquina (ex. Ansible, Puppet).
- Fluxo comum: Terraform cria a VM → Ansible instala e configura o software.
- As duas camadas são complementares, não concorrentes.

As Duas Camadas, Lado a Lado

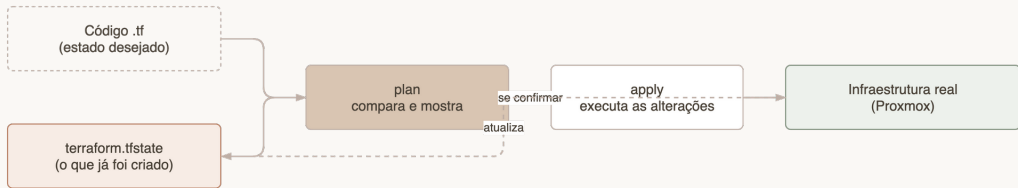


Fluxo típico: Terraform cria a máquina → Ansible configura o que corre lá dentro.

Terraform: o Ficheiro de Estado

- O **state file** regista o mapeamento entre os recursos descritos no código e os recursos reais criados.
- É o que permite ao Terraform saber, em cada execução, o que já existe, o que falta criar e o que deve destruir.
- Em equipa, o estado deve ser guardado num **backend remoto** partilhado (não em ficheiro local), com **locking** para evitar execuções simultâneas.
- O ficheiro de estado pode conter dados sensíveis em texto simples – deve ser tratado como um segredo.
- **Drift**: alterações feitas manualmente fora do Terraform passam a divergir do estado registado, e serão revertidas na aplicação seguinte.

O Ciclo Plan / Apply



Alterações feitas à mão fora do Terraform divergem do estado e são revertidas na aplicação seguinte.

Infraestrutura Mutável vs Imutável

- **Mutável:** os servidores existentes são atualizados no lugar (*in-place*), acumulando alterações ao longo do tempo.
- Risco: divergência progressiva entre máquinas – os chamados *snowflake servers*, difíceis de reproduzir.
- **Imutável:** nunca se altera um servidor em produção; constrói-se uma nova imagem e substitui-se o servidor inteiro.
- Vantagem: cada implantação parte de um estado conhecido e reproduzível; o rollback é trocar de volta para a imagem anterior.
- Os contentores (Aula 04) são o exemplo mais puro deste modelo: a imagem é fixa, e atualizar significa substituir.

Gestão de Segredos

- Problema central do IaC: o código vai para o Git, mas passwords, chaves e tokens não podem ir com ele.
- **Ansible Vault**: cifra ficheiros de variáveis, permitindo guardá-los no repositório em segurança.
- **HashiCorp Vault** e serviços equivalentes: armazenam segredos centralmente, fornecendo-os às ferramentas apenas em tempo de execução.
- Variáveis de ambiente injetadas pelo sistema de CI/CD, nunca escritas no repositório.
- Regra prática: se um segredo alguma vez foi commitado, deve considerar-se comprometido e ser rodado.

CI/CD para Infraestrutura

- Aplicar os mesmos princípios de integração e entrega contínua ao código de infraestrutura.
- Pipeline típico: validar sintaxe → `plan`/dry-run → revisão → `apply` automático.
- Testes automatizados de infraestrutura antes de qualquer mudança chegar a produção.
- Reduz o risco de mudanças manuais não controladas (“shadow IT” de infraestrutura).

GitOps

- Evolução do CI/CD para infraestrutura: o repositório Git passa a ser a *única fonte de verdade* sobre o estado desejado.
- Um agente instalado no ambiente alvo observa o repositório e reconcilia continuamente a realidade com o que lá está declarado.
- Toda a alteração passa por um *commit* e por revisão em *pull request*: fica registada, auditável e reversível.
- Modelo **pull** em vez de **push**: o ambiente vai buscar a configuração, em vez de a receber do exterior – vantajoso em termos de segurança.
- Ferramentas comuns no ecossistema Kubernetes: Argo CD e Flux.

Síntese da Aula (1/2)

- DevOps nasce da tensão entre Dev (mudar depressa) e Ops (manter estável); automatização é a resposta prática a essa tensão.
- Configuração manual leva a drift, erros e falta de reprodutibilidade.
- IaC trata a infraestrutura como código: versionado, idempotente, repetível.
- Ansible (agentless, SSH, YAML) vs Puppet/Chef (agent-based) – cada um com o seu compromisso entre simplicidade e controlo.

Síntese da Aula (2/2)

- Terraform provisiona recursos de forma declarativa (providers, state, plan/apply); o ficheiro de estado é crítico e sensível.
- Provisionamento (que recursos existem) vs gestão de configuração (o que corre neles) – camadas complementares.
- Infraestrutura imutável substitui em vez de alterar; segredos nunca vão para o repositório em texto simples.
- CI/CD e GitOps automatizam teste e aplicação de mudanças, com o Git como fonte de verdade.

Obrigada.

Questões e comentários